

AlphaRAG – Retrieval-Augmented Generation System

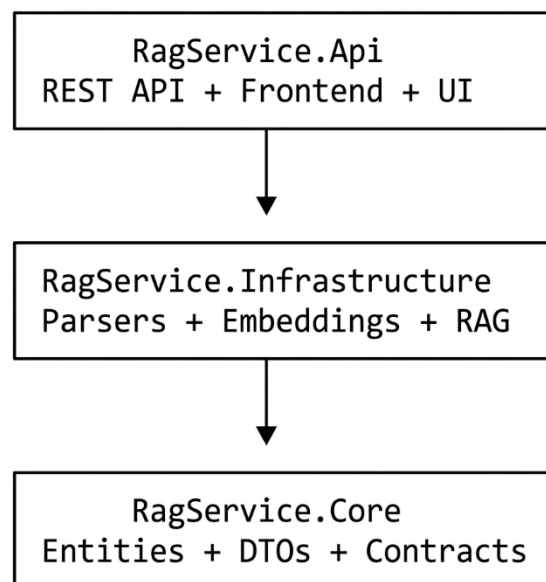
1. Introduction

AlphaRAG is a document-based AI chatbot developed using ASP.NET Core Web API and a modern HTML, CSS, and JavaScript frontend. The system enables users to upload documents, generate semantic embeddings, store them in a vector database, and retrieve relevant information through natural language conversations.

The primary goal of AlphaRAG is to provide accurate, context-aware responses by combining semantic search with Large Language Models (LLMs) such as Ollama and OpenAI.

2. System Architecture

AlphaRAG follows a three-layer architecture to ensure maintainability, scalability, and separation of concerns.



Components

Core Layer

- Domain Models
- DTOs

- Interfaces
- Business Contracts

Infrastructure Layer

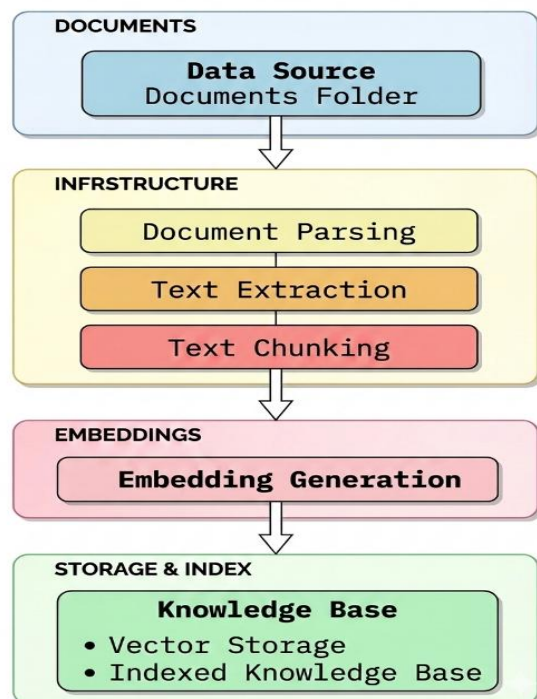
- Document Parsers
- Embedding Services
- Vector Store
- OpenAI/Ollama Providers

API Layer

- REST Endpoints
 - Swagger UI
 - Frontend Hosting
 - Request Processing
-

3. Data Ingestion Workflow

The ingestion process converts uploaded documents into searchable vector embeddings.



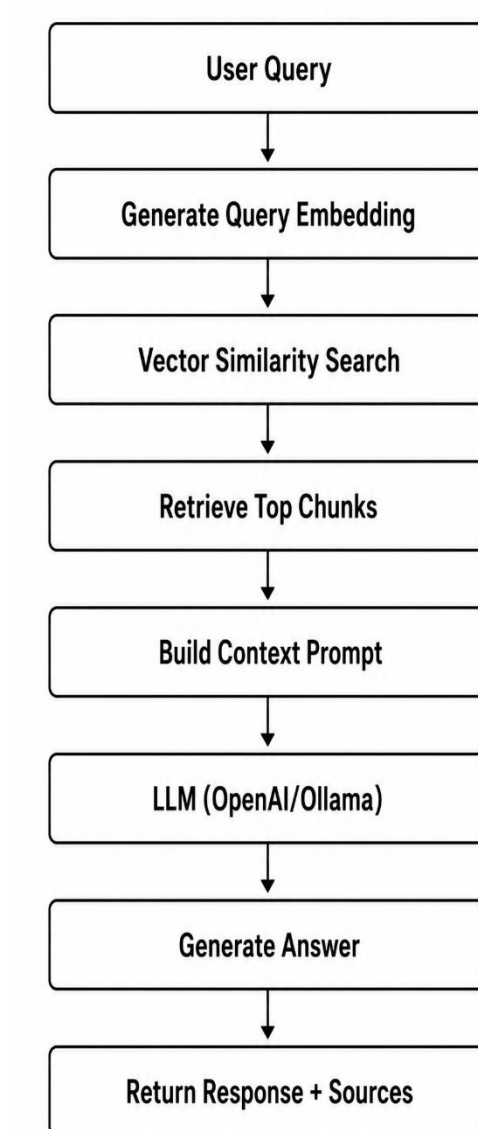
Workflow Explanation

1. Documents are scanned from the configured folder.
2. Text is extracted using format-specific parsers.

3. Large content is divided into chunks.
 4. Embeddings are generated using AI models.
 5. Vectors are stored in memory.
 6. Documents become searchable.
-

4. Chat Retrieval Workflow

When a user asks a question, AlphaRAG retrieves relevant content and generates an answer.



Workflow Explanation

- Query embedding is generated.

- Similar chunks are retrieved using cosine similarity.
 - Retrieved content is passed to the LLM.
 - AI generates a contextual answer.
 - Sources are attached for verification.
-

5. Technology Stack

Component	Technology
Backend	ASP.NET Core 8
Language	C#
Frontend	HTML, CSS, JavaScript
PDF Parsing	PdfPig
Excel Parsing	ExcelDataReader
Word/PPT Parsing	OpenXML
AI Models	Ollama / OpenAI
Vector Search	Cosine Similarity
Storage	In-Memory Vector Store
API Testing	Swagger UI

6. API Endpoints

Health Check

GET /health

Returns application status and configuration information.

Sample Response

```
{  
  "status": "Healthy",  
  "llmProvider": "Ollama",  
  "embeddingProvider": "Ollama"  
}
```

Document Ingestion

POST /ingest

Triggers document indexing and vector generation.

Sample Response

```
{  
  "message": "Ingestion completed successfully"  
}
```

Chat Endpoint

POST /chat

Processes user queries and returns AI-generated answers.

Sample Request

```
{  
  "message": "What services are available?",  
  "limit": 3  
}
```

Sample Response

```
{  
  "answer": "Generated response",  
  "sources": ["document.pdf"]  
}
```

7. Frontend Features

Chat Interface

- Real-time AI conversations
- Source references
- Markdown rendering

Chat History

- Multiple conversations
- Auto-generated titles
- Session persistence using localStorage

Recent Searches

- Stores last five searches
- Quick access through search chips